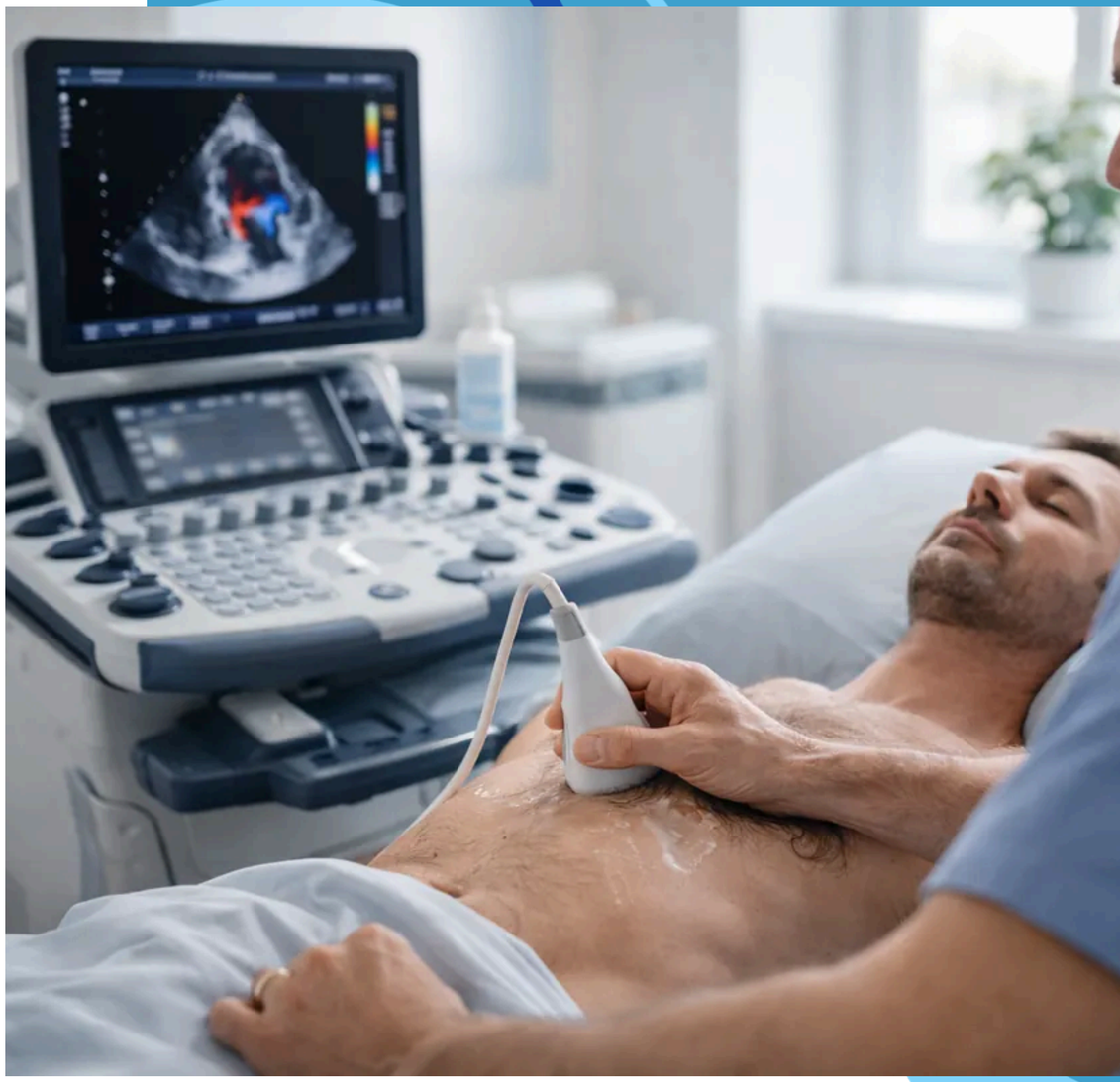
भारतीय सूचना प्रौद्योगिकी संस्थान, पुणे
Indian Institute of Information Technology Pune
(An Institute of National Importance by an Act of Parliament)
Talegaon, Pune, Maharashtra - 410507

B.Tech Project Presentation

FPGA-Accelerated Standalone Echocardiography System for Left Ventricle Segmentation and Ejection Fraction Estimation



Team Members:

Shreyash Jaiswal
Prakrati Yadav
Vivekananda
Sarvesh Kumar Kanth
Zambre Mauli

Faculty Advisor:

Sushant Sir

External Advisor:

Rahman Sir

PROBLEM STATEMENT OVERVIEW

The Problem

- Accurate EF estimation depends on reliable Left Ventricle (LV) segmentation.
- Manual boundary tracing is time-consuming and subjective.
- A lightweight, standalone solution is needed for automated boundary extraction and measurements.

Key Challenges

- Speckle noise and low image contrast.
- Weak or unclear endocardial boundaries.
- Shadowing and missing image information.
- Limited hardware resources for real-time processing.

Core Objectives

- Extract the LV cavity boundary reliably from echocardiography images.
- Estimate LV area and volume from the extracted boundary.
- Support accurate Ejection Fraction (EF) calculation.
- Implement an FPGA-accelerated, hardware-efficient processing pipeline

WORK COMPLETED IN PREVIOUS PHASE

- Reviewed research papers on echocardiography, LV segmentation, FCM and FRFCM.
- Implemented and tested the image-processing pipeline in MATLAB.
- Explored FRFCM-based segmentation, image cleaning and LV boundary extraction.
- Developed a geometry-assisted approach for LV boundary modelling.
- Defined the PS–PL architecture and data flow for FPGA implementation.
- Set up GitHub documentation for project workflow and implementation progress.

https://github.com/Sarveshkk05/EchoFCM-FPGA

PROGRESS ON PROPOSED FUTURE WORK

- **Progress on Proposed Future Work**

1. FPGA-Based Image Display
Successfully demonstrated image display from the FPGA to an HDMI monitor using the ZedBoard.
Investigated the hardware configuration and image-display pipeline using Vivado.

2. Median Filter Implementation
Explored median filtering to reduce noise in echocardiographic images.
Worked on the 3 × 3 median-filter architecture and its integration into the FPGA image-processing pipeline.

3. SD Card Image Input
Investigated reading image data from an SD card.
Worked on integrating image input with the HDMI display pipeline.

4. MATLAB Image Processing
Explored image preprocessing and enhancement techniques.
Investigated FRFCM-based segmentation for identifying the left ventricular (LV) cavity.

5. ECG and Echo-Frame Visualization
Developed Python-based visualization tools to inspect echocardiographic frames, timestamps, and associated ECG information.

- **Remaining Work**

1. Complete and verify median-filter integration with the image input and output pipeline.
2. Validate LV segmentation against suitable reference annotations.
3. Investigate FPGA resource utilization and processing latency.
4. Extend the validated pipeline toward LV measurements and ejection fraction (EF) estimation.

- O**verall Progress**: Established the FPGA-to-HDMI display foundation and explored the software and image-processing components required for the proposed standalone echocardiography system**.**

PROJECT JOURNEY: WHAT WE WORKED ON

- **Echocardiographic Image Preprocessing**
  Image enhancement and median filtering for improved image quality.
- **FPGA Hardware Setup and Display Experiments**
  Configuring the FPGA platform and conducting image-display experiments.
- **FPGA-to-HDMI Image Display**
  Successfully demonstrating image output from the FPGA to an HDMI display.
- **SD Card Image Access and Integration**
  Implementing and integrating image-reading functionality from an SD card.
- **MATLAB-Based Image Processing and GUI Development**
  Developing image-processing workflows and a graphical user interface for experimentation.
- **Dataset Exploration and FRFCM Segmentation**
  Exploring echocardiography datasets and investigating Fuzzy Rule-Based C-Means (FRFCM) segmentation.
- **Python-Based ECG and Echo-Frame Visualisation**
  Developing tools to inspect ECG signals, echocardiographic frames, and temporal information.
- **Hardware–Software Integration and Remaining Challenges**
  integration issues, debugging the pipeline, and identifying the next steps.

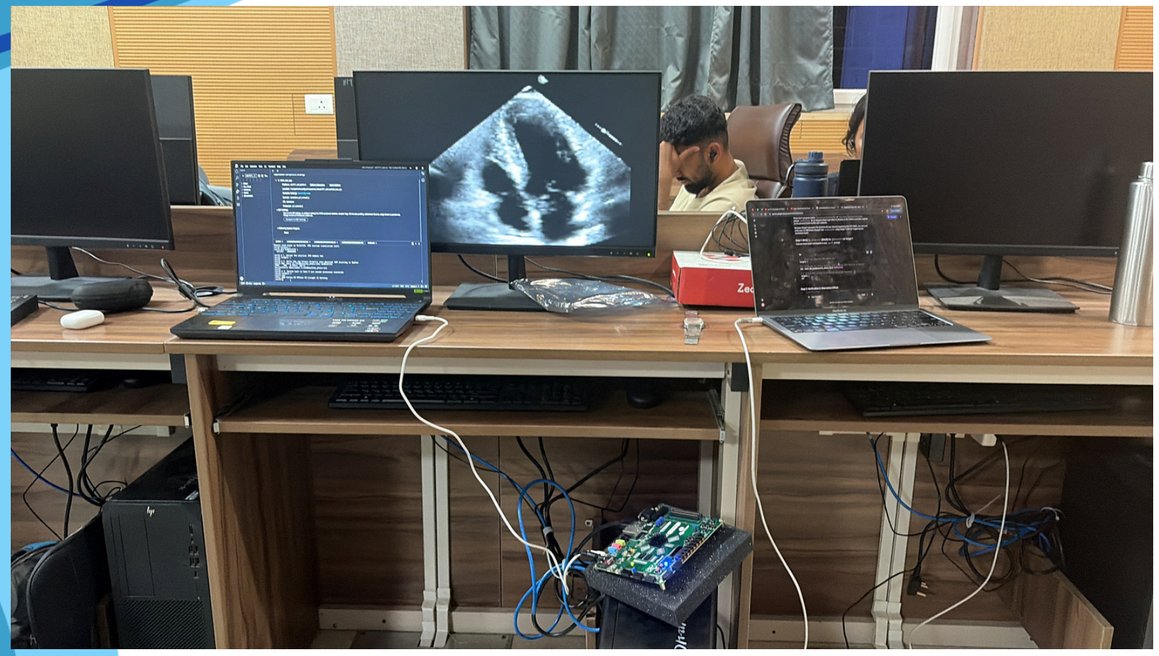
FPGA-TO-HDMI IMAGE DISPLAY: A MAJOR MILESTONE

- **Objective** - Establish a working image-display path using the FPGA

- **Implementation**
  1. Hardware Setup: Configured the FPGA board and connections.
  2. Image Pipeline: Prepared image data for the display path.
  3. Vivado Design: Set up the hardware design and configuration.

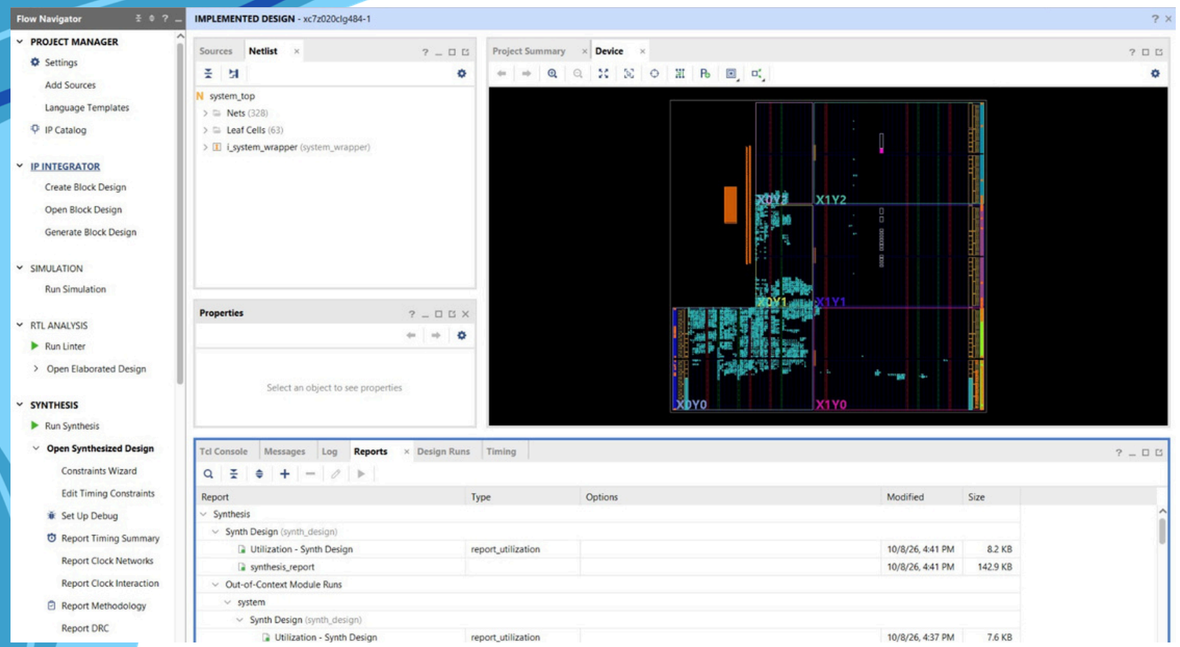
- **Debugging and Demonstration**
  1. Configuration Checks: Investigated setup and integration issues.
  2. HDMI Output: Checked the image on the connected display.
  3. Working Demonstration: Demonstrated the complete image-display path.
  4. Result
  5. Successfully demonstrated FPGA-to-HDMI image display.

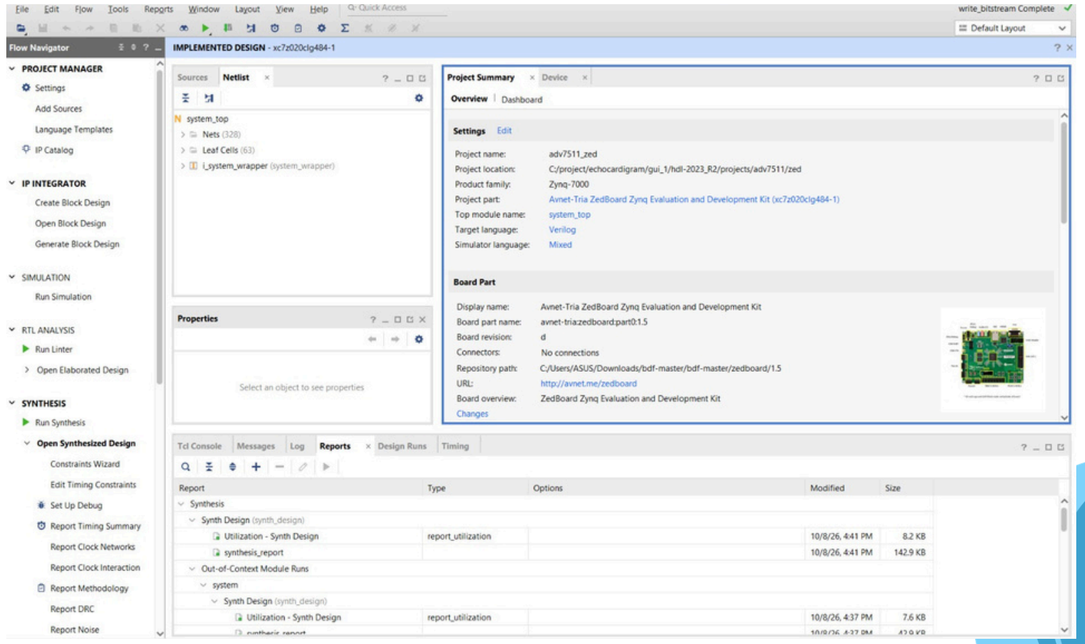
FPGA-TO-HDMI IMAGE DISPLAY: A MAJOR MILESTONE

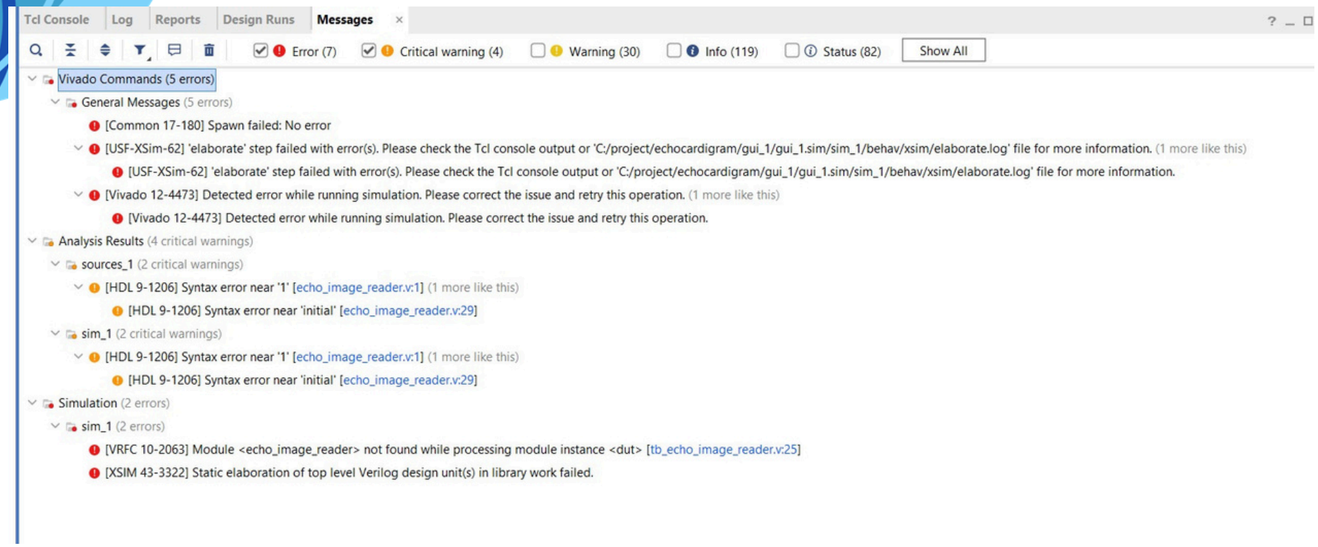


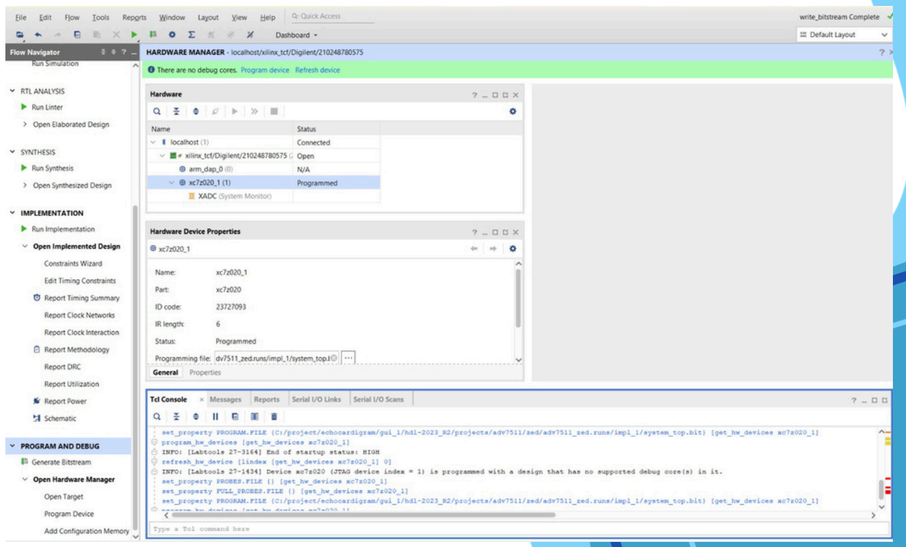


FPGA-to-HDMI Image Display

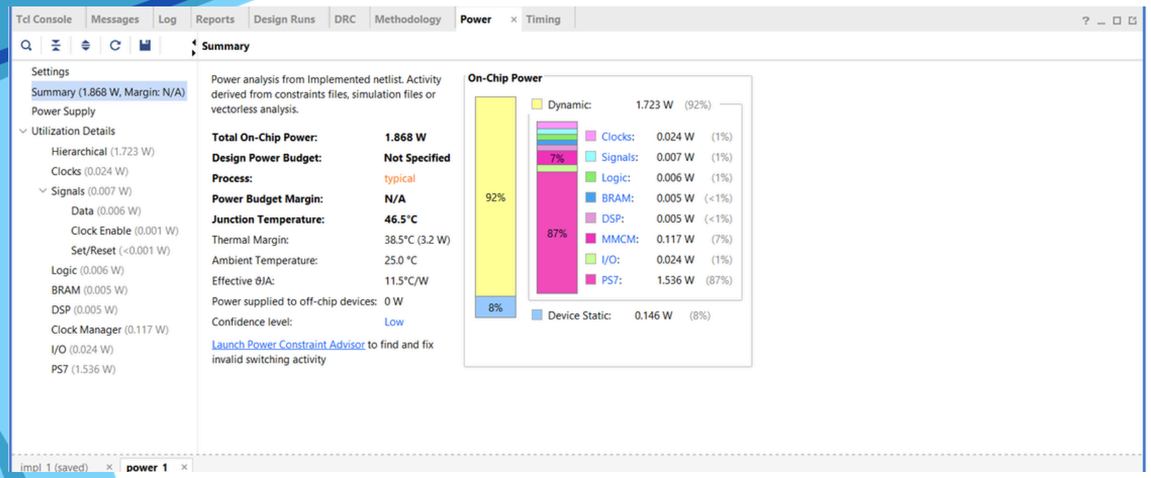


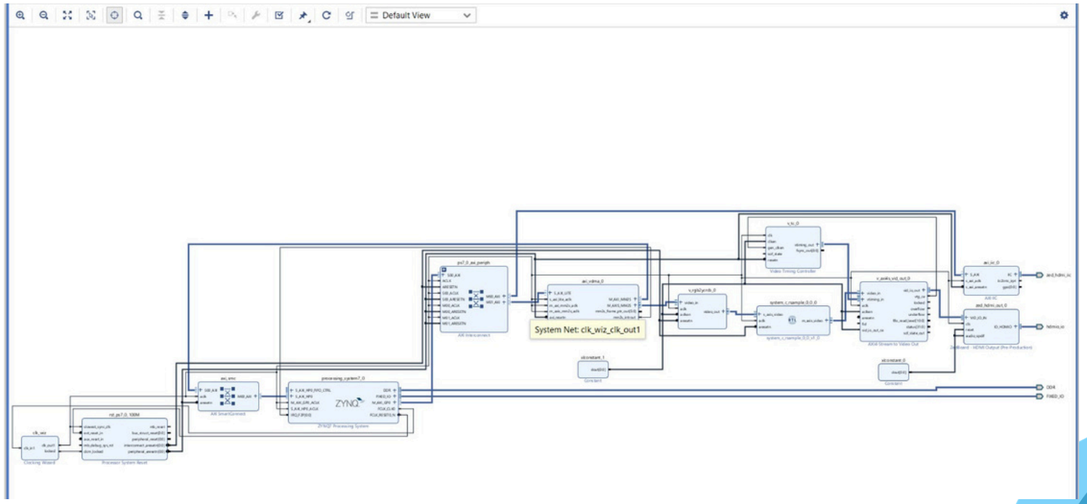


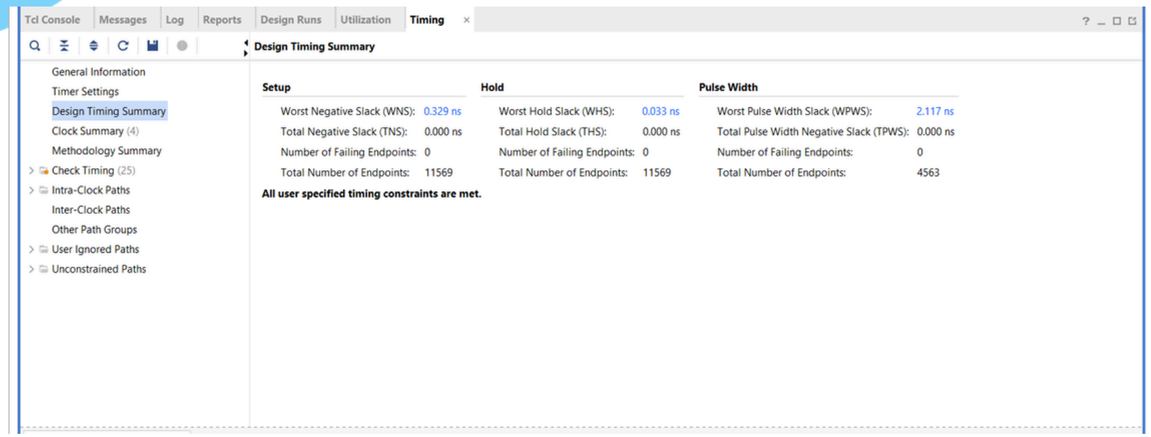




SD card-HDMI


Tcl Console
Messages
Log
Reports
Design Runs
DRC
Methodology
Power
Timing
Summary
Settings
Summary (1.868 W, Margin: N/A)
Power Supply
Utilization Details
Hierarchical (1.723 W)
Clocks (0.024 W)
Signals (0.007 W)
Data (0.006 W)
Clock Enable (0.001 W)
Set/Reset (<0.001 W)
Logic (0.006 W)
BRAM (0.005 W)
DSP (0.005 W)
Clock Manager (0.117 W)
I/O (0.024 W)
PS7 (1.536 W)
Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.
Total On-Chip Power: 1.868 W
Design Power Budget: Not Specified
Process: typical
Power Budget Margin: N/A
Junction Temperature: 46.5°C
Thermal Margin: 38.5°C (3.2 W)
Ambient Temperature: 25.0 °C
Effective θJA: 11.5°C/W
Power supplied to off-chip devices: 0 W
Confidence level: Low
Launch Power Constraint Advisor to find and fix invalid switching activity
On-Chip Power
Dynamic: 1.723 W (92%)
Clocks: 0.024 W (1%)
Signals: 0.007 W (1%)
Logic: 0.006 W (1%)
BRAM: 0.005 W (<1%)
DSP: 0.005 W (<1%)
MMCM: 0.117 W (7%)
I/O: 0.024 W (1%)
PS7: 1.536 W (87%)
Device Static: 0.146 W (8%)
92%
7%
87%
8%
impl_1 (saved)
power 1


Default View
System Net: clk_wiz_clk_out1


Tcl Console
Messages
Log
Reports
Design Runs
Utilization
Timing
Design Timing Summary
General Information
Timer Settings
Design Timing Summary
Clock Summary (4)
Methodology Summary
Check Timing (25)
Intra-Clock Paths
Inter-Clock Paths
Other Path Groups
User Ignored Paths
Unconstrained Paths
Setup
Worst Negative Slack (WNS): 0.329 ns
Total Negative Slack (TNS): 0.000 ns
Number of Failing Endpoints: 0
Total Number of Endpoints: 11569
Hold
Worst Hold Slack (WHS): 0.033 ns
Total Hold Slack (THS): 0.000 ns
Number of Failing Endpoints: 0
Total Number of Endpoints: 11569
Pulse Width
Worst Pulse Width Slack (WPWS): 2.117 ns
Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0
Total Number of Endpoints: 4563
All user specified timing constraints are met.


Tcl Console
Messages
Log
Reports
Design Runs
Utilization
Timing
Hierarchy
Hierarchy
Summary
Slice Logic
Slice LUTs (7%)
LUT as Memory (2%)
LUT as Shift Register
LUT as Distributed
LUT as Logic (6%)
F8 Muxes (<1%)
F7 Muxes (<1%)
Slice Registers (4%)
Register as Flip Flop
Memory
Block RAM Tile (4%)
RAMB18 (1%)
RAMB18E1 only
RAMB36/FIFO (3%)
RAMB36E1 only
Name
Slice LUTs (53200)
Slice Registers (106400)
F7 Muxes (26600)
F8 Muxes (13300)
Block RAM Tile (140)
DSPs (220)
Bonded IOB (200)
Bonded IOPADs (130)
OLOGIC (200)
BUFGCTRL (32)
MMCME2_ADV (4)
system_wrapper 3546 4015 9 1 5 5 23 130 1 3 1
system_i (system) 3546 4015 9 1 5 5 0 0 1 3 1
utilization 1

FPGA Implementation – Median Filter +SD card +HDMI

Tcl Console | Messages | Log | Reports | Design Runs | Timing | Power | DRC | Noise

Design Timing Summary

- General Information
- Timer Settings
- Design Timing Summary
- Clock Summary (10)
- Methodology Summary
- Check Timing (37)
- Intra-Clock Paths
- Inter-Clock Paths
- Other Path Groups
- User Ignored Paths
- Unconstrained Paths

Setup		Hold		Pulse Width	
Worst Negative Slack (WNS):	-24.942 ns	Worst Hold Slack (WHS):	0.045 ns	Worst Pulse Width Slack (WPWS):	0.090 ns
Total Negative Slack (TNS):	-1596.039 ns	Total Hold Slack (THS):	0.000 ns	Total Pulse Width Negative Slack (TPWS):	0.000 ns
Number of Failing Endpoints:	64	Number of Failing Endpoints:	0	Number of Failing Endpoints:	0
Total Number of Endpoints:	342814	Total Number of Endpoints:	342814	Total Number of Endpoints:	169720

Timing constraints are not met.

Tcl Console | Messages | Log | Reports | Design Runs | Power

Summary

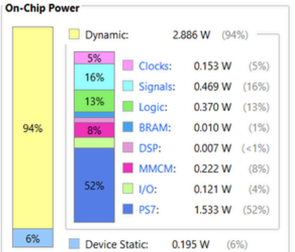
Settings
Summary (3.082 W, Margin: N/A)
Power Supply
Utilization Details
Hierarchical (2.886 W)
Clocks (0.153 W)
Signals (0.469 W)
Data (0.467 W)
Clock Enable (0.002 W)
Set/Reset (<0.001 W)
Logic (0.37 W)
BRAM (0.01 W)
DSP (0.007 W)
Clock Manager (0.222 W)
I/O (0.121 W)
PS7 (1.533 W)

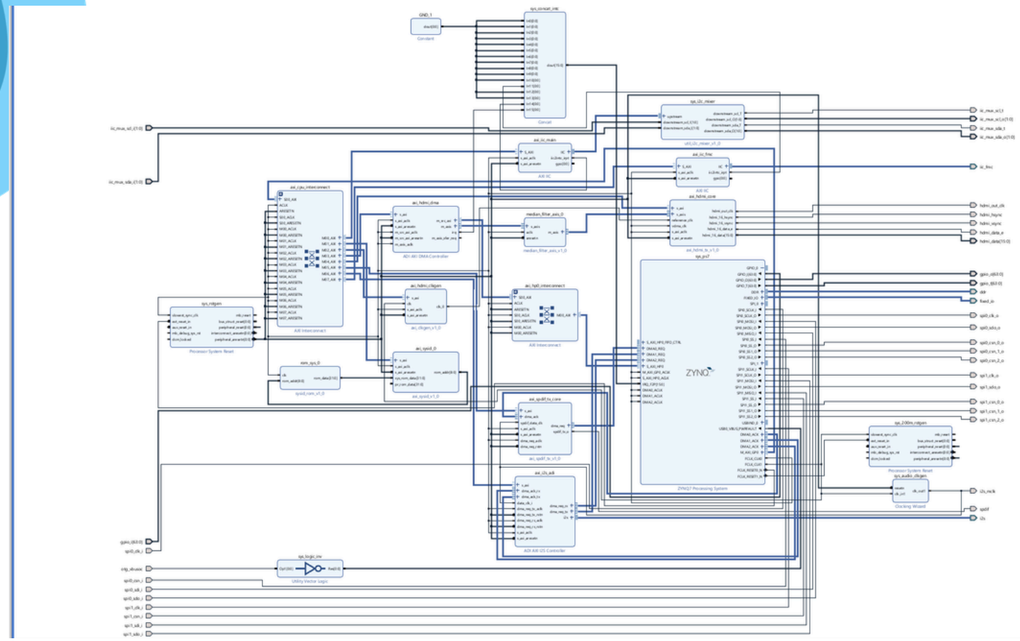
Power estimation from Synthesized netlist. Activity derived from constraints files, simulation files or vectorless analysis. Note: these early estimates can change after implementation.

Total On-Chip Power:	**3.082 W**
Design Power Budget:	**Not Specified**
Process:	typical
Power Budget Margin:	**N/A**
Junction Temperature:	**60.5°C**
Thermal Margin:	24.5°C (2.0 W)
Ambient Temperature:	25.0 °C
Effective θJA:	11.5°C/W
Power supplied to off-chip devices:	0 W
Confidence level:	Low

Launch Power Constraint Advisor to find and fix invalid switching activity

On-Chip Power





Tcl Console | Messages | Log | Reports | Design Runs | DRC | Methodology | Power | Timing | Utilization

Hierarchy

Hierarchy
Summary
Slice Logic
Slice LUTs (<1%)
LUT as Logic (<1%)
Slice Registers (<1%)
Register as Flip Flop (<
Slice Logic Distribution
Slice (<1%)
SLICEL
SLICEM
Slice Registers (<1%)
Register driven from wi
Register driven from ou
LUT in front of the
LUT in front of the
LUT as Logic (<1%)
using O6 output only<
using O5 and O6<>T a

Name	Slice LUTs (53200)	Slice Registers (106400)	Slice (13300)	LUT as Logic (53200)	Bonded IOB (200)	BUFGCTRL (32)
video_top	34	50	24	34	28	1
pattern_inst (test_pattern)	1	17	10	1	0	0
timing_inst (video_timing)	33	33	18	33	0	0

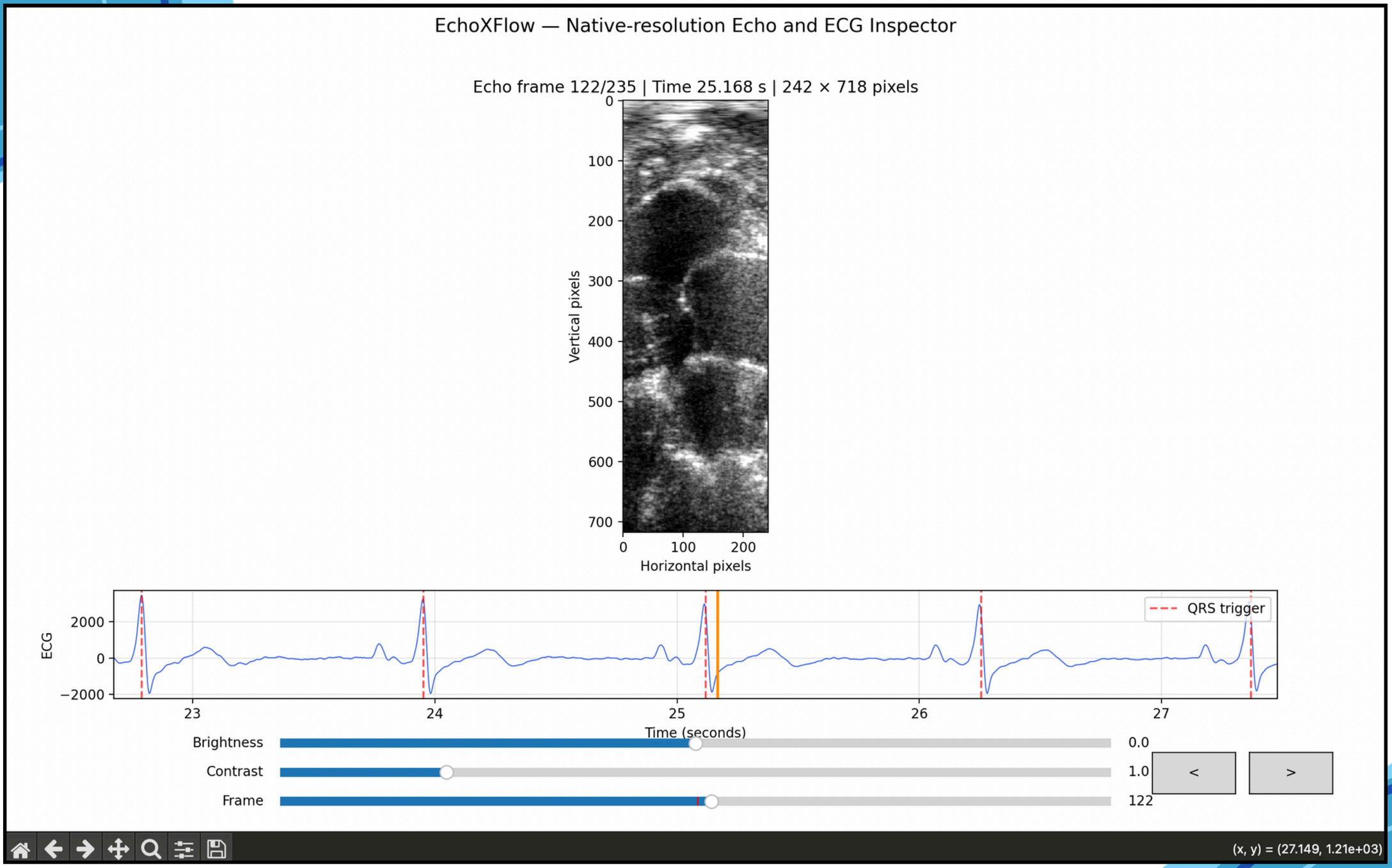
utilization_1

ECG AND ECHOCARDIOGRAPHY FRAME VISUALISATION

1. Python-Based Viewer
 Developed a Python viewer to inspect echocardiography recordings and their associated data.
2. Echo-Frame Visualisation
 Displayed individual echocardiographic frames to examine cardiac structures and LV visibility.
3. Timestamp Inspection
 Inspected frame timestamps to understand temporal information and frame sequencing.
4. ECG and QRS-Marker Visualisation
 Visualised ECG signals and QRS markers where the required data and implementation were available.
5. Cardiac Cycle Analysis
 Explored how cardiac timing can help relate electrical activity to ventricular contraction and relaxation.

Outcome

Established a visualisation workflow for examining echocardiographic frames, temporal information, and associated cardiac electrical activity.


EchoXFlow — Native-resolution Echo and ECG Inspector
Echo frame 122/235 | Time 25.168 s | 242 × 718 pixels
Vertical pixels
Horizontal pixels
ECG
QRS trigger
Time (seconds)
Brightness
Contrast
Frame
0.0
1.0
122
<
>

ECHOCARDIOGRAPHY DATASET EXPLORATION

Datasets and their purpose

- EchoXFlow-Demo: Investigating echo recordings and associated ECG/timing information.
- EchoNet-Dynamic: Exploring echocardiography videos for left ventricular (LV) analysis.
- CAMUS: Examining annotated LV contours for segmentation evaluation.

Practical challenges

- LV visibility: Not all recordings clearly show the LV cavity.
- Image quality: High resolution alone does not guarantee useful anatomical detail.
- Data handling: Differences in file formats and recording structures can complicate processing.
- Evaluation: Reliable validation requires suitable ground-truth contours or reference measurement

FPGA PROJECTS AND REPOSITORIES WORKED WITH AND EXPLORED

Reference Projects Explored

1. GJigar/HDMI-display

Purpose: SD Card Image and Video Handling

Explored reading raw video frames and image assets directly from an SD card using FATFS (xilffs) on the ZedBoard.

2. megalloid/Zynq-7000-PL-HDMI-and-VDMA-IP-setup

Purpose: HDMI Display Pipeline

Used as a reference for configuring AXI Video Direct Memory Access (VDMA) and the ADV7511 HDMI transmitter for video output.

3. USTC-SSE-practice/FPGA-Image-Processing-2

Purpose: Hardware-Accelerated Image Processing

Explored reference implementations of image-filtering pipelines using custom High-Level Synthesis (HLS) cores and AXI DMA data transfers.

Key Takeaway

These projects provided reference implementations and technical guidance for understanding SD card access, FPGA memory transfers, hardware-accelerated image processing, and HDMI display integration.

WHY FPGA DEVELOPMENT REQUIRED TIME

Design and Data setup

- Hardware Configuration
- Configuring the FPGA design and connecting the required interfaces.
- Data Representation
- Understanding how image pixels stored, transferred, and interpreted.
- Memory & Interfaces
- Managing data transfer between memory and processing components

Integration and testing

- Display Integration
- Moving image data through the hardware pipeline to the HDMI output.
- Debugging
- Tracing errors in configuration, data handling, and hardware integration.
- Repeated Testing
- Modifying the design, rebuilding it, and checking fixes on the FPGA board.

REFERENCES

- Ouyang et al. (2020). Video-Based AI for Beat-to-Beat Assessment of Cardiac Function. Nature. https://doi.org/10.1038/s41586-020-2145-8
- Leclerc et al. (2019). Deep Learning for Segmentation Using an Open Large-Scale Dataset in 2D Echocardiography. IEEE TMI. https://doi.org/10.1109/TMI.2019.2900516
- Lei et al. (2018). Significantly Fast and Robust Fuzzy C-Means Clustering Algorithm Based on Morphological Reconstruction and Membership Filtering. IEEE TFS. https://doi.org/10.1109/TFUZZ.2018.2796074
- Babu & Parthasarathy (2021). Hardware Acceleration of Image and Video Processing on Xilinx Zynq Platform. https://doi.org/10.32604/iasc.2021.018903
- AMD/Xilinx. Zynq-7000 SoC Technical Documentation. https://docs.amd.com/
- MathWorks. Default Video System Reference Design for Zynq-7000. //www.mathworks.com/help/hdlcoder/xilinxzynq7000/ug/default-video-system-reference-design.html
- MathWorks. Contrast-Limited Adaptive Histogram Equalization (CLAHE). https://www.mathworks.com/help/images/adaptive-histogram-equalization.html
- MathWorks. Median Filtering. https://www.mathworks.com/help/images/ref/medfilt2.html
- Sobel Edge Detection. https://www.mathworks.com/help/images/ref/edge.html
- AMD/Xilinx. Zynq-7000 Partial Reconfiguration Reference Design. https://xilinx-wiki.atlassian.net/wiki/spaces/A/pages/18842428/Zynq+7000+Partial+Reconfiguration+Reference+Design
- EchoNet-Dynamic. Official Research Code and Dataset Documentation. https://github.com/echonet/dynamic
- MathWorks. FPGA-Based Image Processing with External Memory. https://www.mathworks.com/help/soc/ug/contrast-adaptive-histogram-equalization-ext-mem.html

CONCLUSION

- Hardware Achievement: Successfully demonstrated FPGA-to-HDMI image display using a Vivado-based design on the ZedBoard.
- Algorithmic Exploration: Investigated echocardiographic image preprocessing and FRFCM-based segmentation for potential LV cavity identification.
- Software Development: Explored echocardiography datasets and developed Python-based visualisation tools for echo frames, timestamps, and associated ECG information.
- Challenges Identified: LV visibility, image noise, data handling, hardware integration, and the need for ground-truth-based validation.
- Future Direction: Implement and optimise LV segmentation on the FPGA, evaluate segmentation accuracy, and investigate validated LV area and ejection fraction estimation.

Final Outcome: Established the hardware-display foundation and explored the algorithmic and software components needed to advance towards FPGA-based echocardiographic LV analysis.

THANK YOU